Model Predictive Control

Lecture Handout

April 16, 2026

Contents

Part I

Introduction

Chapter 1

Introduction to Model Predictive Control

1.1 Optimization-Based Control

A central idea behind model predictive control is that the control action is not generated by a fixed pre-computed feedback law alone, but by repeatedly solving an optimization problem online. This point of view is best understood by comparing it with the classical control loop. In a standard feedback architecture, the controller receives a reference and the measured output of the plant, and then applies an input according to a control law that has been designed in advance. In optimization-based control, this fixed controller block is replaced, at least conceptually, by an optimizer.

The optimizer uses a model of the plant to predict how the system will evolve under candidate future inputs. Based on these predictions, it computes an input sequence that best satisfies the desired control objectives while respecting the relevant constraints. The first important shift is therefore methodological: instead of asking directly for a controller formula, we formulate a decision problem whose solution determines the input to be applied.
This approach is particularly natural when the control objective is tightly coupled with physical limitations and safety requirements. Consider, for instance, the problem of driving a race car around a track. One may wish to minimize the lap time, but this objective cannot be pursued arbitrarily. The car must stay on the road, avoid collisions, respect limitations on tire friction, and remain within admissible acceleration and steering capabilities. The control task is therefore inherently constrained. A meaningful strategy is to look ahead, predict the upcoming road geometry and the effect of future decisions, and then plan a feasible trajectory that achieves the best possible performance. This is precisely the kind of reasoning that optimization-based control formalizes.

The same idea can be described algorithmically as follows. At a given time instant, one solves an optimization problem to compute a planned trajectory and a corresponding sequence of future control actions. From this sequence, only the first control move is actually implemented. At the next sampling instant, the state is measured again, the prediction is updated, and a new optimization problem is solved. In this way, planning and feedback are combined in a single framework.

Optimization-based control

The key idea is to compute the control input by solving, at each sampling instant, a finite-dimensional optimization problem built from three ingredients: a performance objective, a

predictive model of the plant, and a set of constraints.

1.2 Concept of Model Predictive Control

Model predictive control, usually abbreviated as MPC, is the most prominent realization of optimization-based control. The name itself summarizes its defining features. It is *model-based* because future behavior is predicted through an internal model of the plant. It is *predictive* because the controller explicitly reasons over a future horizon. Finally, it is a *control* method because the resulting optimization problem is solved repeatedly in closed loop in order to decide the plant input.

A useful way to understand MPC is to compare it with more classical control design philosophies. In classical control, one typically designs a controller C for a plant P so as to achieve good disturbance rejection, limited sensitivity to measurement noise, and robustness to modeling uncertainty. The dominant language is often that of transfer functions, loop shaping, and frequency-domain specifications. By contrast, MPC is naturally expressed in the **time domain**. Its strength lies in the **explicit treatment of constraints**, both on the manipulated inputs and on the system variables. This makes it especially appealing in applications where safety, actuator saturation, and operating envelopes are of central importance.
Indeed, all physical systems are constrained. Inputs cannot be arbitrarily large because actuators have limits. Outputs and states are often restricted by performance or safety considerations, such as maximum temperature, pressure, velocity, or overshoot. In many practical settings, the economically or operationally optimal regime lies close to such limits. Traditional control methods often handle constraints only indirectly, for example by tuning the set-point conservatively or by adding ad hoc supervisory logic. MPC instead incorporates the constraints directly into the control design. This allows operation closer to the admissible boundaries, while still maintaining systematic decision-making.

Mathematically, MPC can be formulated over a finite prediction horizon of length N. At a generic time instant k, one considers the current state $x(k)$ and searches for a sequence of future control inputs

$$U_k = \{u_k, u_{k+1}, \dots, u_{k+N-1}\}$$

that minimizes a cumulative performance criterion. A standard formulation is

$$U_k^\star(x(k)) := \arg\min_{U_k} \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i})$$

subject to the system dynamics

$$x_{k+i+1} = Ax_{k+i} + Bu_{k+i},$$

the initial condition

$$x_k = x(k),$$

and the constraints

$$x_{k+i} \in \mathcal{X}, \qquad u_{k+i} \in \mathcal{U},$$

for all relevant indices i.
This compact expression already contains the essential ingredients of MPC. The objective function defines what the controller tries to optimize. The model provides predictions of future behavior. The constraint sets $\mathcal{X}$ and $\mathcal{U}$ encode admissible operating conditions. The decision variables are the future inputs across the prediction horizon.

The online operation of MPC follows a receding-horizon principle. At each sampling time, the current state is measured or estimated, the finite-horizon optimization problem is solved, and an optimal input sequence

$$U_k^\star = \{u_k^\star, u_{k+1}^\star, \dots, u_{k+N-1}^\star\}$$

is obtained. However, only the first element $u_k^\star$ is applied to the plant. After one sampling interval, the state is updated and the entire optimization is solved again from the new initial condition. The horizon therefore moves forward in time, which is why one also speaks of a *receding horizon* strategy.
This repeated re-optimization is what **introduces feedback**. If the plant is disturbed, if the model is imperfect, or if the measured state differs from the previously predicted one, the next optimization step automatically adjusts the future plan. The controller does not blindly execute an open-loop plan; instead, it continuously replans using fresh information.

Receding horizon principle

MPC computes an *optimal sequence* of future control moves, but implements only the *first* one. Feedback arises because the optimization problem is solved again at the next sampling instant using the newly measured or estimated state.

1.2.1 Why constraints matter

The explicit inclusion of constraints is one of the main reasons for the success of MPC. Suppose one wants to regulate a process output to a desired set-point while an upper safety limit must never be violated. A classical controller may achieve fast convergence but produce overshoot that crosses the admissible bound. To avoid this, one may reduce aggressiveness or move the set-point away from the constraint, which typically leads to suboptimal operation. MPC offers a more systematic alternative: the constraint is included directly in the optimization problem, and the computed control action is chosen with full awareness of that limit.
This ability to reason ahead is crucial. The optimizer can anticipate that a constraint will become active in the near future and can begin adjusting the input before the violation occurs. In this sense, MPC is predictive not only in the use of a model, but also in the way it handles constraints proactively rather than reactively.

1.3 Applications of MPC

One of the remarkable features of MPC is the **breadth of its application domain**. The same conceptual framework appears in problems evolving at extremely different time scales.

At very fast time scales, optimization-based control ideas arise in computer and electronic systems, where decisions may need to be made on the **nanosecond or microsecond** level. Slightly slower, but still extremely fast, are power system applications, where control actions and operational decisions can be taken in the microsecond range depending on the problem formulation and hardware layer under consideration.
At the **millisecond** scale, traction control and automotive systems provide a classical application area. Here, one must coordinate performance objectives with safety and actuator limits in real time. On the scale of **seconds**, building control becomes relevant: thermal dynamics are slow enough to allow relatively long prediction horizons, yet constraints and energy-efficiency objectives play a major role. At the scale of **minutes and hours**, refinery operation and process control have historically been among the main industrial drivers for MPC. At even slower scales, **days or weeks**, one finds planning and scheduling problems such as train scheduling, nurse

rostering, and production planning, where the same optimization-and-prediction philosophy remains useful.

These examples show that MPC should not be viewed only as a single algorithm for fast dynamical systems. Rather, it is a general methodology for repeated decision-making under dynamical models and constraints. The exact mathematical models, optimization problems, and computational requirements vary greatly across applications, but the underlying structure remains the same.

1.4 A Brief History of MPC

The historical roots of MPC go back to the early idea of using optimization methods directly inside control algorithms. An early milestone is the work of A. I. Propoi in 1963, where linear programming ideas were proposed for sampled-data automatic systems. This already contains the seed of a now-familiar idea: control can be posed as the repeated solution of an optimization problem.
The modern industrial development of MPC is commonly traced to the 1970s. A key step was the work of Richalet and co-authors in 1978 on what became known as IDCOM, short for *Identification and Command*. Their approach used an impulse response model of the plant, considered a finite prediction horizon, and formulated a quadratic performance objective. Future plant behavior was described relative to a desired reference trajectory, and practical input/output constraints were included in an ad hoc manner. Because the resulting controller was not expressed as a conventional transfer function but rather as the outcome of an iterative computational procedure, the approach was described as heuristic predictive control. At this stage, the framework was mainly aimed at stable plants and was strongly motivated by industrial needs.

A closely related and highly influential development came from Cutler and collaborators. In the late 1970s, Cutler proposed ideas that were later implemented at Shell under the name *Dynamic Matrix Control* (DMC). The 1979 work by Cutler and Ramaker is particularly important in the industrial history of MPC. DMC used a step response model of the plant, a finite-horizon quadratic objective, and a prediction mechanism that aimed at tracking the set-point as closely as possible. The optimal manipulated variables were computed through a least-squares problem, and additional equations were introduced online to account for constraints. The name "dynamic matrix" refers precisely to the structured matrix that arises in this least-squares formulation.
A further important step occurred in 1983, when Cutler, Morshedi, and Haydel emphasized the formulation of the controller as a standard quadratic program. This was conceptually decisive, because it provided a systematic mathematical framework for handling constraints rather than relying only on heuristic modifications. From that point on, MPC gradually evolved from an industrial engineering practice into a discipline with a firm optimization-theoretic foundation.

In the mid 1990s, much theoretical work was devoted to understanding under which conditions one can guarantee recursive feasibility and closed-loop stability. This period established many of the fundamental results that are now part of the standard theory of nominal MPC. During the 2000s, research expanded toward tractable robust MPC methods, nonlinear MPC, hybrid systems, and fast MPC algorithms tailored to increasingly demanding real-time requirements. In the 2010s, important developments included stochastic MPC, distributed and large-scale MPC, and economic MPC, where the objective is not merely set-point tracking but direct optimization of process performance. In the 2020s, a major direction has been the interaction between MPC and learning, leading to a growing body of work on learning-based MPC.

1.5 Summary and Main Aspects

Model predictive control can therefore be summarized as a control methodology in which, at every sampling instant, one measures or estimates the current state, predicts future behavior with an internal model, solves a constrained finite-horizon optimization problem, and applies only the first input of the optimal sequence. The procedure is then repeated at the next sampling instant.

Its main appeal lies in the fact that performance objectives and operating constraints are treated within a single optimization-based framework. This gives the designer a systematic way to handle actuator limits, safety requirements, and multi-variable interactions while maintaining a feedback structure through repeated re-planning.
At the same time, MPC also comes with **important challenges**. First, the optimization problem must be solved reliably and fast enough for real-time use. Second, the finite-horizon problem may become infeasible if the constraints are too restrictive or if the system is driven into an unfavorable region. Third, closed-loop stability is not automatic and must typically be enforced or analyzed through suitable design choices. Finally, the closed-loop behavior may be sensitive to model mismatch and disturbances, which motivates the study of robustness and the many extensions of the nominal MPC framework.

Chapter 2

Systems theory basics

2.1 Models of Dynamic Systems

Model predictive control relies on a mathematical model of the system to predict future behavior. This is one of the fundamental ingredients of the entire framework: without a model, there is no basis for forecasting the effect of future control inputs and therefore no basis for optimization-based planning. In practice, one also needs a state estimator, a suitable optimal control problem formulation, a numerical optimization procedure, and finally a verification that the resulting closed-loop system behaves as desired. In this sense, system theory provides the language and tools on which MPC is built.

Dynamic-system models can be classified in several ways. One may distinguish between state-space and transfer-function descriptions, between linear and nonlinear models, between time-varying and time-invariant dynamics, between continuous-time and discrete-time formulations, and between deterministic and stochastic models. In this handout, as in the lecture, the focus is mainly on deterministic models. Physical systems derived from first principles are typically described by nonlinear, time-invariant, continuous-time state-space models, whereas the standard models used in MPC are most often linear, time-invariant, discrete-time state-space models. A substantial part of the modeling workflow therefore consists in transforming the former into the latter in a meaningful way.

2.1.1 Nonlinear continuous-time state-space models

A very general class of dynamical systems can be written in **continuous-time state-space form** as

$$\dot{x} = g(x, u), \qquad y = h(x, u),$$

where $x \in \mathbb{R}^n$ is the state vector, $u \in \mathbb{R}^m$ is the input, and $y \in \mathbb{R}^p$ is the output. The map $g : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ describes the system dynamics, while $h : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^p$ specifies which quantities are measured or observed. This representation is extremely flexible and can describe a very broad range of physical systems.
An important reason why the state-space formalism is so useful is that higher-order ordinary differential equations can always be rewritten as systems of first-order equations. If one starts from an nth-order differential equation

$$x^{(n)} + g_n(x, \dot{x}, \ddot{x}, \ldots, x^{(n-1)}) = 0$$

suitable state variables can be introduced by stacking the variable and its derivatives up to order $n - 1$. This is possible by defining

$$x_{j+1} = x^{(j)}, \qquad j = 0, 1, \ldots, n - 1,$$

where $x^{(0)}$ is simply x. The original equation can then be rewritten as

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \\ -g_n(x_1, x_2, \ldots, x_n) \end{bmatrix}.$$

The original equation is then transformed into n coupled first-order equations. This conversion is not merely formal: it provides a standard representation in which analysis and controller design can be carried out systematically.

The **drawback** of such nonlinear descriptions is that, although they are general and physically meaningful, their analysis and controller synthesis are often difficult. For this reason, one often replaces them locally by linear approximations around a chosen operating point.

2.1.2 Linear time-invariant continuous-time models

A continuous-time linear time-invariant system is written as

$$\dot{x} = A_c x + B_c u, \qquad y = Cx + Du,$$

where A_c, B_c, C, and D are constant matrices of compatible dimensions. Compared with the nonlinear case, this model class is much more **restrictive**, but it has a decisive **advantage**: a vast mathematical theory is available for its analysis and control design.
One important feature of linear systems is that their solution can be written explicitly. If

$$\dot{x}(t) = A_c x(t) + B_c u(t), \qquad x(t_0) = x_0,$$

then the state at time t is given by

$$x(t) = e^{A_c(t-t_0)} x_0 + \int_{t_0}^{t} e^{A_c(t-\tau)} B_c u(\tau)\, d\tau,$$

where the matrix exponential is defined by

$$e^{A_c t} := \sum_{n=0}^{\infty} \frac{(A_c t)^n}{n!}.$$

This expression makes the effect of the initial condition and the input signal fully explicit.

2.1.3 Linearization around an operating point

Most real systems are nonlinear, yet linear models are much easier to analyze and use in controller design. The standard compromise is therefore to **linearize the nonlinear dynamics around an operating point**. The idea is that if the system is to operate near a desired equilibrium, then a first-order approximation may be sufficiently accurate in a neighborhood of that point.

The mathematical basis is the first-order Taylor expansion. If a nonlinear function is expanded around a point $\bar{x}$, then near $\bar{x}$ it can be approximated by its value at $\bar{x}$ plus the Jacobian times the deviation from that point. The first order Taylor approximation of a function $f(\cdot)$ around $\bar{x}$ is

$$f(x) \approx f(\bar{x}) + \frac{\partial f}{\partial x^\top}\bigg|_{x=\bar{x}} (x - \bar{x} \qquad \text{with} \frac{\partial f}{\partial x^\top} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

Applied to a nonlinear state-space system

$$\dot{x} = g(x,u), \qquad y = h(x,u),$$

and linearized around a stationary operating point (x_s, u_s) satisfying

$$g(x_s, u_s) = 0,$$

one obtains a linear model in deviation variables:

$$\dot{x} = \underbrace{g(x_s,u_s)}_{=0} + \underbrace{\frac{\partial g}{\partial x^\top}\Big|_{x=x_s,u=u_s}}_{=A_c} \underbrace{(x-x_s)}_{\Delta x} + \underbrace{\frac{\partial g}{\partial u^\top}\Big|_{x=x_s,u=u_s}}_{=B_c} \underbrace{(u-u_s)}_{\Delta u}$$

Thus

$$\dot{x} - \underbrace{\dot{x}_s}_{=0} = \Delta\dot{x} = A_c\Delta x + B_c\Delta u$$

Proceeding in the same way for the output equation, one obtains

$$\Delta y = \underbrace{\frac{\partial h}{\partial x^\top}\Big|_{x=x_s,u=u_s}}_{=C} \underbrace{(x-x_s)}_{\Delta x} + \underbrace{\frac{\partial h}{\partial u^\top}\Big|_{x=x_s,u=u_s}}_{=D} \underbrace{(u-u_s)}_{\Delta u}$$

Thus we obtain the linearized model

$$\Delta\dot{x} = A_c\Delta x + B_c\Delta u, \qquad \Delta y = C\Delta x + D\Delta u,$$

where

$$A_c = \frac{\partial g}{\partial x^\top}\Big|_{x=x_s,u=u_s}, \qquad B_c = \frac{\partial g}{\partial u^\top}\Big|_{x=x_s,u=u_s},$$

$$C = \frac{\partial h}{\partial x^\top}\Big|_{x=x_s,u=u_s}, \qquad D = \frac{\partial h}{\partial u^\top}\Big|_{x=x_s,u=u_s}.$$

After linearization, it is common to drop the Δ notation and simply denote deviations again by x, u, and y.

The pendulum example clearly illustrates both the usefulness and the limitations of this procedure. Around a chosen equilibrium, such as a constant nonzero angle with zero angular velocity, the nonlinear sine term is replaced by its local linear approximation. The resulting matrices depend on the selected operating point. The linearized dynamics provide a good approximation only in a small neighborhood: for small angles and velocities the behavior of the linear and nonlinear systems is close, whereas farther away the approximation deteriorates. This local nature of linearization is essential to keep in mind in MPC modeling.

Linearization

Linearization replaces a nonlinear model by a local linear approximation around a stationary operating point. It is extremely useful for analysis and design, but its validity is inherently local: it can only be trusted in a sufficiently small neighborhood of the linearization point.

2.1.4 Discrete-time state-space models

Since controllers are typically implemented on digital hardware, MPC is naturally formulated in discrete time. A **discrete-time system** is described by difference equations of the form

$$x(k+1) = g(x(k), u(k)), \qquad y(k) = h(x(k), u(k)),$$

where the input and output are defined only at discrete instants indexed by $k \in \mathbb{Z}_+$. Some systems are inherently discrete, such as an account balance updated every month, but most control systems originate in continuous time and are then discretized for implementation.
The sampling instants are typically of the form

$$t_{k+1} = t_k + T_s,$$

where T_s is the sampling time. A standard assumption is that the control input is held constant between two consecutive sampling instants,

$$u(t) = u(t_k), \qquad t \in [t_k, t_{k+1}),$$

which is the usual zero-order-hold implementation.

2.1.5 Euler discretization

For nonlinear systems, one simple discretization method is the forward Euler approximation. Starting from the continuous-time model

$$\dot{x}_c(t) = g_c(x_c(t), u_c(t)), \qquad y_c(t) = h_c(x_c(t), u_c(t)),$$

one approximates the derivative by

$$\dot{x}_c(t) \approx \frac{x_c(t+T_s) - x_c(t)}{T_s}.$$

where T_s is the sampling time. Introducing the notation

$$x(k) = x_c(t_0 + kT_s), \qquad u(k) = u_c(t_0 + kT_s),$$

one obtains the discrete-time approximation

$$x(k+1) = x(k) + T_s g_c(x(k), u(k)), \qquad y(k) = h_c(x(k), u(k)).$$

Under suitable regularity assumptions and for sufficiently small sampling times, the discrete-time trajectory approximates the continuous-time one well.

For linear continuous-time systems,

$$\dot{x}(t) = A_c x(t) + B_c u(t), \qquad y(t) = C_c x(t) + D_c u(t),$$

Euler discretization yields

$$x(k+1) = Ax(k) + Bu(k), \qquad y(k) = Cx(k) + Du(k),$$

with

$$A = I + T_s A_c, \qquad B = T_s B_c, \qquad C = C_c, \qquad D = D_c.$$

This is straightforward and often useful, although it is not exact.

2.1.6 Exact discretization of linear systems

For linear systems under zero-order hold, one can derive an exact discrete-time model. Using the explicit continuous-time solution and assuming that the input remains constant over one sampling interval, one obtains

$$x(t_{k+1}) = e^{A_c T_s} x(t_k) + \left(\int_0^{T_s} e^{A_c(T_s-\tau)} B_c \, d\tau \right) u(t_k).$$

Hence the exact discrete-time system is

$$x(k+1) = Ax(k) + Bu(k),$$

with

$$A = e^{A_c T_s}, \qquad B = \int_0^{T_s} e^{A_c(T_s-\tau)} B_c \, d\tau.$$

If A_c is invertible, then B can also be written as

$$B = A_c^{-1}(A - I)B_c.$$

This model is **exact** in the sense that it predicts the state of the continuous-time system exactly at the sampling instants, provided the zero-order-hold assumption is satisfied.

2.1.7 Solution of linear discrete-time systems

One major advantage of discrete-time linear systems is that their future state can be written explicitly as a function of the initial condition and the input sequence. Starting from

$$x(k+1) = Ax(k) + Bu(k),$$

successive substitution gives

$$x(k+N) = A^N x(k) + \sum_{i=0}^{N-1} A^i B \, u(k+N-1-i).$$

Thus, for fixed matrices A and B, the predicted state over a horizon is a **linear function of the current state and of the future inputs**. This observation is fundamental in MPC, because it allows the finite-horizon optimization problem to be written directly in terms of the decision variables and the known initial state.

The modeling discussion can be summarized as follows. Continuous-time nonlinear models are general and physically natural, but they are difficult to analyze and use directly in predictive control. Linearization gives a local continuous-time approximation around an operating point. Discretization then converts the model into a form that is directly compatible with digital control implementation. For linear systems, exact zero-order-hold discretization is available, while for general nonlinear systems one often relies on approximate methods such as Euler discretization, whose quality depends strongly on the sampling time. These are precisely the model transformations that make standard MPC formulations possible.

2.2 Analysis of LTI Discrete-Time Systems

Once a model has been put into discrete-time LTI form, one can study several structural properties that are central in estimation and control. Among the most important are the behavior of the system under coordinate changes, asymptotic stability, controllability, and observability. These notions form the classical backbone of linear systems theory and play a direct role in MPC.

2.2.1 Coordinate transformations

Consider the discrete-time LTI system

$$x(k+1) = Ax(k) + Bu(k), \qquad y(k) = Cx(k) + Du(k).$$

The input-output behavior is completely determined by the initial state and the input sequence. However, the choice of state variables is **not unique**. Different state representations may generate exactly the same input-output behavior, and some representations may be more convenient than others for analysis or design.
If one introduces a new state variable

$$\tilde{x}(k) = Tx(k),$$

where T is invertible, then the transformed system becomes

$$\tilde{x}(k+1) = \tilde{A}\tilde{x}(k) + \tilde{B}u(k), \qquad y(k) = \tilde{C}\tilde{x}(k) + \tilde{D}u(k),$$

with

$$\tilde{A} = TAT^{-1}, \qquad \tilde{B} = TB, \qquad \tilde{C} = CT^{-1}, \qquad \tilde{D} = D.$$

The input and output remain unchanged; only the internal coordinates have changed. Such transformations are useful because they can simplify the structure of the matrices and thereby reveal important properties more transparently.

2.2.2 Stability of linear discrete-time systems

For the autonomous system

$$x(k+1) = Ax(k),$$

asymptotic stability means that every trajectory converges to the origin as $k \to \infty$. A fundamental theorem states that this happens if and only if all eigenvalues of A lie strictly inside the unit circle, that is,

$$|\lambda_j| < 1, \qquad j = 1, \ldots, n.$$

This is the discrete-time analogue of the continuous-time condition $\mathrm{Re}(\lambda_j) < 0$.

The intuition becomes especially clear when the system can be diagonalized. In suitable coordinates, the system matrix becomes diagonal, and each transformed state component evolves independently as multiplication by the corresponding eigenvalue. If the magnitude of every eigenvalue is less than one, repeated multiplication drives each component to zero. If any eigenvalue has magnitude greater than or equal to one, some component fails to decay. Even when diagonalization is not possible and Jordan forms must be used, the same conclusion remains valid.

2.2.3 Controllability and stabilizability

A system

$$x(k+1) = Ax(k) + Bu(k)$$

is **controllable** if, for any initial state $x(0)$ and any desired target state $x*$, there exists a finite time N and an input sequence $\{u(0), u(1), \ldots, u(N-1)\}$ that steers the system exactly to that target, i.e. $x(N) = x*$. In discrete time, this notion is often also called reachability. By expanding the state recursively:

$$x* = x(N) = A^N x(0) + \sum_{i=0}^{N-1} A^i B u(N-1-i) = A^N x(0) + \begin{bmatrix} B & AB & \cdots & A^{N-1}B \end{bmatrix} \begin{bmatrix} u(N-1) \\ u(N-2) \\ \vdots \\ u(0) \end{bmatrix}$$

It follows from the **Carley-Hamilton** theorem that A^k can be expressed as a linear combination of I, A, $\ldots$, A^{n-1}, so that for all $N \geq n$

$$\text{range}\begin{bmatrix} B & AB & \cdots & A^{N-1}B \end{bmatrix} = \begin{bmatrix} B & AB & \cdots & A^{n-1}B \end{bmatrix}$$

We therefore conclude that the reachable states are precisely those that can be generated by the columns of the **controllability matrix**

$$\mathcal{C} = \begin{bmatrix} B & AB & \cdots & A^{n-1}B \end{bmatrix},$$

this means that the reachable states are precisely those in the column space of $\mathcal{C}$. The system is controllable **if and only if**

$$\text{rank}(\mathcal{C}) = n.$$

A weaker notion is **stabilizability**. A system is stabilizable if the unstable part of the dynamics can still be influenced by the input, even if some already stable modes are uncontrollable. More precisely, the system is stabilizable if all uncontrollable modes are stable. Controllability always implies stabilizability, but the converse need not hold. In many feedback-design settings, stabilizability is the essential requirement.
Stabilizability can be checked using the following condition

$$\text{if rank}\Big(\begin{bmatrix} \lambda_j I - A | B \end{bmatrix}\Big) = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, B) \text{is stabilizable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

2.2.4 Observability and detectability

Dual to controllability is the notion of **observability**. Consider the zero-input system

$$x(k+1) = Ax(k), \qquad y(k) = Cx(k).$$

The system is observable if the initial state can be uniquely reconstructed from a finite sequence of output measurements. By stacking the outputs over time, one obtains the linear relation

$$\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(n-1) \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} x(0).$$

The matrix

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

is called the **observability matrix**, and the system is observable **if and only if**

$$\text{rank}(\mathcal{O}) = n.$$

Again, the use of only the first n blocks is justified by the Cayley–Hamilton theorem.

The weaker property corresponding to stabilizability is **detectability**. A system is detectable if all unobservable modes are stable, so that although the state may not be reconstructed exactly, the estimation error can still be driven to zero asymptotically. Observability implies detectability,

but not every detectable system is observable. In practice, detectability is often the minimal property needed to construct a convergent observer or state estimator.
Observability can be checked using the following condition

$$\text{if rank}\left(\begin{bmatrix}\lambda_j I - A \\ C\end{bmatrix}\right) = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, C) \text{ is detectable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

Structural properties

Controllability and observability are exact structural properties of a model. Their relaxed counterparts, stabilizability and detectability, are often sufficient for controller and observer design and are especially relevant when only the unstable modes must be influenced or reconstructed.

2.3 Analysis of Nonlinear Discrete-Time Systems

The previous section dealt with discrete-time LTI systems, for which many results can be stated in terms of matrices and eigenvalues. For nonlinear systems, the situation is more subtle. Stability is no longer merely a property of the matrix A, but a **property of an equilibrium point** of the nonlinear dynamics. The appropriate conceptual framework is Lyapunov stability theory.

2.3.1 Lyapunov stability of nonlinear systems

Consider an autonomous nonlinear discrete-time system

$$x(k+1) = g(x(k)),$$

and let $\bar{x}$ be an equilibrium point, meaning that

$$g(\bar{x}) = \bar{x}.$$

Lyapunov stability formalizes the intuitive idea that if the system is perturbed only slightly from equilibrium, then it should remain close to equilibrium for all future time. More precisely, $\bar{x}$ is Lyapunov stable if for every $\varepsilon > 0$ there exists a $\delta(\varepsilon) > 0$ such that

$$\|x(0) - \bar{x}\| < \delta(\varepsilon) \quad \Longrightarrow \quad \|x(k) - \bar{x}\| < \varepsilon \qquad \forall k \geq 0.$$

If, in addition, trajectories converge to the equilibrium (i.e. is attractive), meaning that

$$\lim_{k\to\infty} \|x(k) - \bar{x}\| = 0 \qquad \forall x(0) \in \mathbb{R}^n,$$

then the equilibrium is **globally asymptotically stable**.

2.3.2 Lyapunov functions

A standard way to prove stability is to construct a Lyapunov function. The intuition comes from mechanics: if the total energy of a damped system decreases over time, then the system tends to settle to an equilibrium. Lyapunov functions generalize this idea to arbitrary dynamical systems. For an equilibrium at the origin, a **Lyapunov function** is a scalar function

$$V : \mathbb{R}^n \to \mathbb{R}$$

that is continuous at the origin, finite everywhere, radially unbounded, positive definite, and strictly decreases along system trajectories. In particular, one requires

$$V(0) = 0, \qquad V(x) > 0 \text{ for all } x \neq 0,$$

$$\|x\| \to \infty \implies V(x) \to \infty,$$

and

$$V(g(x)) - V(x) \leq -\alpha(x),$$

where α is a continuous positive definite function. **If such a function exists, then the origin is globally asymptotically stable**. If the decrease condition holds only with α positive semidefinite, then one obtains Lyapunov stability but not necessarily asymptotic convergence.

2.3.3 Indirect Lyapunov method

Although Lyapunov functions are powerful, they are often **difficult to construct** for general nonlinear systems. A useful alternative is the indirect Lyapunov method, which studies the linearization around the equilibrium. Let

$$A = \left.\frac{\partial g}{\partial x^\top}\right|_{x=0},$$

then the origin is **locally asymptotically stable if** all eigenvalues of A satisfy

$$|\lambda_i| < 1.$$

On the other hand, if at least one eigenvalue satisfies

$$|\lambda_i| > 1,$$

then the origin is unstable. The borderline case in which some eigenvalue lies exactly on the unit circle is **inconclusive**, and one must resort to a more refined analysis, often through a Lyapunov function.

This theorem is especially important because it connects nonlinear local stability with the familiar eigenvalue test from linear systems. It also clarifies why linearization is so useful: it can certify local asymptotic stability when the equilibrium lies safely inside the stable region, but it cannot always settle the question in marginal cases.

2.3.4 Lyapunov stability of linear systems

For **linear systems**, Lyapunov theory becomes constructive. Consider

$$x(k+1) = Ax(k).$$

A natural candidate Lyapunov function is quadratic,

$$V(x) = x^\top P x,$$

with $P = P^\top > 0$. Evaluating the change along trajectories gives

$$V(x(k+1)) - V(x(k)) = x(k)^\top (A^\top P A - P) x(k).$$

If one can choose $P > 0$ such that

$$A^\top P A - P = -Q, \qquad Q > 0, \tag{2.1}$$

then

$$V(x(k+1)) - V(x(k)) = -x(k)^\top Q x(k) < 0$$

for all nonzero $x(k)$, which proves asymptotic stability. Equation (2.1) is the **discrete-time Lyapunov equation**.

A central theorem states that, for discrete-time LTI systems, the Lyapunov equation has a **unique positive definite solution** P for every given $Q > 0$ **if and only if** all eigenvalues of A lie inside the unit circle. Thus, in the linear case, eigenvalue stability and the existence of a quadratic Lyapunov function are equivalent. Moreover, because the system is linear, this stability is automatically global (stability is always "global" for linear systems, since the dynamics are the same everywhere in the state space).

There is also an important **interpretation** of the matrix P in terms of infinite-horizon cost. If the system is asymptotically stable and one defines

$$\Psi(x(0)) = \sum_{k=0}^{\infty} x(k)^\top Q x(k),$$

then this infinite sum can be written as

$$\Psi(x(0)) = x(0)^\top P x(0),$$

where P is precisely the solution of the discrete-time Lyapunov equation.
We can proof this by defining $H_k = (A^k)^\top Q (A^k)$ and $P = \sum_{k=0}^{\infty} H_k$. Notice that the limit of the sum exists because the system is asymptotically stable. Then we have that

$$A^\top H_k A = (A^{k+1})^\top Q (A^{k+1}) = H_{k+1}$$

Thus

$$A^\top P A = A^\top \left(\sum_{k=0}^{\infty} H_k \right) A = \sum_{k=0}^{\infty} A^\top H_k A = \sum_{k=0}^{\infty} H_{k+1} = P - H_0 = P - Q$$

In other words, the same quadratic function that certifies stability also represents the infinite-horizon cost-to-go associated with the stage cost $x^\top Q x$. This link is conceptually very important for MPC, because later the optimal cost itself will play the role of a Lyapunov function for the closed-loop system under suitable assumptions.

2.4 Summary

The material of this chapter provides the basic system-theoretic background required for MPC. Starting from nonlinear continuous-time models, one typically linearizes around an operating point and then discretizes the system to obtain a model suitable for digital control implementation. For linear discrete-time systems, explicit predictions over a finite horizon can be written as linear functions of the current state and the future input sequence, which is exactly the structure exploited in predictive control.
On the analysis side, stability of LTI systems is characterized by eigenvalue location, while controllability and observability determine whether the system can be influenced and reconstructed from measurements. Their weaker counterparts, stabilizability and detectability, are often sufficient for design tasks. For nonlinear systems, Lyapunov theory provides the appropriate framework to study stability of equilibria. In the linear case, quadratic Lyapunov functions arise naturally through the discrete-time Lyapunov equation and connect stability analysis with infinite-horizon cost. This bridge between dynamics, stability, and cost is one of the conceptual pillars on which the theory of MPC is built.

Chapter 3

Unconstrained Control

Chapter 4

Introduction to ConvexOptimization

Part II

Nominal MPC

Chapter 5

Constrained Finite Time Optimal Control

Chapter 6

Invariance

Chapter 7

Feasibility and Stability

Part III

Practical MPC

Chapter 8

Practical Issues in MPC

Part IV

Robust MPC

Chapter 9

Robust MPC 1

Chapter 10

Robust MPC 2

Part V

Implementation Methods

Chapter 11

Implementation Methods